AF SECTIONS 1 TO 10

LAYER 3, MACHINE-BINDABLE SPECS

Internal Use Only. Not for external distribution.
Version: L3-v8 — Standalone machine contract for intake feasibility gating.

How to use this document

This is the machine-bindable specification for AF intake.

Layer 1 governs meaning, boundary, posture, and nondisclosure doctrine.
Layer 2 governs what Jules must build, validate, test, and deliver.
Layer 3 governs exact machine expression.

Use this document to answer:

- exact enums
- exact allowed templates
- exact context values
- exact run rules
- exact sufficiency logic
- exact stop logic
- exact sequencing logic
- exact clean-state isolation rules
- exact note permission logic
- exact validation rules for external output
- exact intake record contracts

Where Layer 1 states a doctrinal obligation, Layer 3 defines how that obligation is expressed and enforced in machine terms.

SECTION 1: PURPOSE AND BOUNDARY OF INTAKE [LOCKED]

1.1 Intake phase identity

intake_phase = feasibility_gate is the only permitted phase identity for AF 1 to 10 intake logic.

1.2 Scope anchor input

The only permitted scope-anchor input for intake is:

- complaint or demand materials provided

No alternate scope source may create intake eligibility behavior unless governance updates the doctrine.

1.3 Prohibited intake-purpose modes

The intake system must not expose any mode, label, record, output, or workflow that represents intake as:

- compliance audit
- defect inventory
- remediation guide
- certification
- legal opinion

1.4 External output boundary

External-facing intake output is limited to eligibility determination and doctrine-permitted mechanically neutral scope and context wording. No internal execution depth or internal selection detail may be exposed.

SECTION 2: AUTHORITY, SOURCE OF TRUTH, AND DRIFT CONTROL [LOCKED]

2.1 Implementation source of truth

Repository-implemented AFDM intake behavior is the source of truth for:

- output labels
- record fields
- parameter locks
- constraint handling
- stop logic
- template usage

2.2 Conflict behavior

If doctrine text and implemented intake behavior conflict, runtime implementation behavior governs what the system actually did. That does not authorize silent drift. The conflict must be surfaced and governed correction required.

Drift events are not captured in the intake record schemas defined in Appendix A. Drift detection, reporting, and correction are governed externally to the intake record set and are not validated at this layer.

2.3 Prohibited internal evaluative fields

Internal intake records must not include:

- evaluative fields
- intent signals
- strength indicators
- probability indicators
- outcome-leaning metadata

2.4 Prohibited internal posture framing

Internal intake record fields and outcome categories must not imply:

- compliance posture
- liability posture
- likelihood posture

2.5 Prohibited internal framing terms

Internal intake records must not use record terminology that frames entries as:

- findings
- violations
- assessments

2.6 No improvisation rule

No new labels, fields, procedures, stop rules, or metadata classes may be introduced informally to resolve doctrinal ambiguity.

SECTION 3: MECHANICAL OBSERVER LANGUAGE [LOCKED]

3.1 External banned framing validator surface

External-facing intake output must reject the following terms and functional equivalents:

- pass
- fail
- compliant
- non-compliant
- violation
- audit, as intake purpose
- remediation, as intake purpose
- certification
- guarantee
- blame posture
- adversarial posture

For purposes of this validator surface, blame posture or adversarial posture means any language that assigns fault, implies intent, suggests wrongdoing, or frames either party as acting improperly.

3.2 Mandatory exact external scope and context terms

When external-facing intake output describes scope and context, it must use these exact terms:

- complaint or demand materials provided
- specific website conditions asserted in those materials
- bounded execution parameters
- Replicated Desktop Browser Context
- Replicated Mobile Browser Context

Paraphrase is not permitted.

3.3 Mechanical neutrality rule

External-facing output must describe only mechanically observed state under bounded execution parameters. Characterizing the site, claim, or either party is prohibited.

3.4 Burden-rule validator surface

External-facing intake output must remain literal about:

- what was executed
- what was observed
- what was blocked by a constraint
- what could not be bounded due to missing specification

Rhetorical burden-shifting language is invalid.

3.5 Indirect-signaling validator examples

External-facing intake output must reject phrases that signal internal execution depth indirectly, including:

- extensive testing
- limited testing
- we checked everything
- we checked only a few items

Functional equivalents are also prohibited, even if the exact phrase does not appear on this list. Functional-equivalent detection requires reviewer judgment and is not fully automatable. This check must be flagged for reviewer clearance and recorded in the ExternalOutputValidationRecord.

SECTION 4: INTAKE DETERMINATION TEMPLATES [LOCKED]

4.1 Exactly one determination per matter

Each matter must produce exactly one external-facing intake determination.

4.2 Locked determination template enum

The only permitted external determination templates are, numbered for reference throughout this document:

- Template 1: DETERMINATION: ELIGIBLE FOR DESKTOP AND MOBILE TECHNICAL RECORD BUILD
- Template 2: DETERMINATION: ELIGIBLE FOR DESKTOP TECHNICAL RECORD BUILD
- Template 3: DETERMINATION: ELIGIBLE FOR DESKTOP TECHNICAL RECORD BUILD / MOBILE BASELINE: CONSTRAINED
- Template 4: DETERMINATION: ELIGIBLE FOR MOBILE TECHNICAL RECORD BUILD
- Template 5: DETERMINATION: ELIGIBLE FOR MOBILE TECHNICAL RECORD BUILD / DESKTOP BASELINE: CONSTRAINED
- Template 6: DETERMINATION: NOT ELIGIBLE FOR FORENSIC EXECUTION
- Template 7: DETERMINATION: NOT ELIGIBLE FOR FORENSIC EXECUTION - CONSTRAINTS (BOTMITIGATION)
- Template 8: DETERMINATION: NOT ELIGIBLE FOR FORENSIC EXECUTION - CONSTRAINTS (OTHER)

No additional determination values are permitted.

4.3 Determination-line nondisclosure rule

The determination line must not disclose, directly or indirectly:

- closeness to sufficiency
- selected or attempted asserted conditions
- observed or not observed results
- run cap reached status

This nondisclosure rule does not prohibit the determination template itself from disclosing matter-level context status, including Desktop-only eligibility, Desktop and Mobile eligibility, Desktop eligibility with Mobile baseline constrained, Mobile-only eligibility, or Mobile eligibility with Desktop baseline constrained.

4.4 Determination-line anti-hedging rule

The determination line must reject modifiers or hedging terms including:

- provisionally
- appears
- strongly
- weakly
- likely

- unlikely

and functional equivalents. Functional-equivalent detection requires reviewer judgment and is not fully automatable. This check must be flagged for reviewer clearance and recorded in the ExternalOutputValidationRecord.

4.5 Peer-baseline eligibility and constraint guardrails

Machine rules:

- Template 1 is the only dual-baseline eligibility template
- Template 2 is the only Desktop-only eligibility template
- Template 3 is the only Desktop-eligible plus Mobile-constrained template
- Template 4 is the only Mobile-only eligibility template
- Template 5 is the only Mobile-eligible plus Desktop-constrained template
- Mobile-preferred outputs are invalid
- Desktop-preferred outputs are invalid
- partial-eligibility outputs are invalid

For purposes of this rule, partial-eligibility means any output that grants eligibility for a baseline not governed by a locked determination template, or that qualifies, limits, or conditions eligibility in a manner not expressed through one of the eight locked determination templates. Templates 3 and 5 are not partial-eligibility outputs; they are the locked constrained-baseline templates and are the only valid expression of single-eligible-baseline plus constrained-peer-baseline status.

4.6 Template 3 and Template 5 conditional validity

Template 3 is valid only if:

- Mobile baseline is in scope for the matter
- Mobile baseline execution is blocked under controlled parameters
- the relevant internal run state includes an applicable locked `constraint_class`

Template 5 is valid only if:

- Desktop baseline is in scope for the matter

- Desktop baseline execution is blocked under controlled parameters
- the relevant internal run state includes an applicable locked constraint_class

For Templates 3 and 5 only, the locked constraint_class value that produced the constraint condition is permitted in external-facing output at the matter level only, using the locked constraint_class enum value. No per-run constraint detail may be disclosed.

4.7 Constraint-template selection for ineligible matters

Template 6, DETERMINATION: NOT ELIGIBLE FOR FORENSIC EXECUTION, is valid only when the matter is not eligible for forensic execution for reasons other than a technical constraint. Template 6 must not be used when the controlling ineligibility basis is a constraint condition. When ineligibility is constraint-driven, Template 7 or Template 8 governs.

Template 7, DETERMINATION: NOT ELIGIBLE FOR FORENSIC EXECUTION - CONSTRAINTS (BOTMITIGATION), is valid only when the controlling ineligibility basis is BOTMITIGATION.

Template 8, DETERMINATION: NOT ELIGIBLE FOR FORENSIC EXECUTION - CONSTRAINTS (OTHER), is valid only when the matter is not eligible for forensic execution due to constraint conditions other than BOTMITIGATION.

Template 7 must not be used for non-BOTMITIGATION ineligibility. Template 8 must not be used when the controlling ineligibility basis is BOTMITIGATION. Template 6 must not be used when any locked constraint_class is the controlling ineligibility basis.

SECTION 5: RUN CAP, SUFFICIENCY, AND STOP RULES [LOCKED]

5.1 Run cap

Maximum runs per matter: 10

No time cap.
No assertion-count cap.
The 10-run cap is the only run-volume bound.

5.2 Run definition

One run equals:

- one controlled attempt
- of one asserted website condition
- under one browser context
- producing exactly one outcome label from the locked intake outcome set

For purposes of this section, a controlled attempt means a Playwright-executed browser session initiated under the locked `ContextProfileRecord` parameters for the declared `context_id`, navigating to the relevant page or state, and evaluating the asserted website condition under clean-state isolation as required by Section 10.3. Playwright is the locked execution framework for this layer and is not treated as implementation-agnostic at Layer 3.

5.3 Qualifying confirmations

The only qualifying confirmation outcomes are, as defined in Section 7.1:

- Observed as asserted
- Not observed as asserted

The following are not qualifying confirmations:

- Constrained
- Insufficiently specified for bounded execution

5.4 Sufficiency threshold

Sufficiency is reached when:

- there are 2 qualifying confirmations
- each confirmation comes from a distinct run

Those confirmations may come from any mix of contexts that are in scope for the matter. For single-baseline matters, both qualifying confirmations must come from the context that is in scope. The two qualifying confirmations may come from runs against the same `run_unit_id` or different `run_unit_id` values; distinct runs are required, not distinct run units.

5.5 Immediate stop rule

Stop immediately when either condition occurs first:

- sufficiency reached
- 10 runs completed

When sufficiency is reached on the same run that completes the 10-run cap, both stop conditions are met simultaneously. In that case, `stop_basis` must be recorded as `sufficiency_reached`.

5.6 No post-stop execution

No additional runs are permitted after the stop condition is met for that matter.

5.7 Intake closure rule

Once a stop condition is met, intake is closed for that matter. Intake must not be resumed or extended without a new matter scope submission.

5.8 Constrained runs consume run cap slots

A run that produces the outcome label `Constrained` or `Insufficiently specified for bounded execution` counts against the 10-run cap. It does not produce a qualifying confirmation. The run cap applies to all runs regardless of outcome label.

SECTION 6: COMPLAINT-ANCHORED NORMALIZATION [LOCKED]

6.1 Complaint group anchor forms

Each complaint or demand grouping must be assigned a `complaint_group_anchor` using one of these forms only:

- `page_paragraph_range`
- `page_bullet_range`

No alternate anchor types are permitted.

6.2 Complaint group anchor is reference only

A complaint_group_anchor is a reference pointer only. It does not determine the number of executable conditions.

6.3 RUN_UNIT atomicity

Each distinct asserted website condition within a complaint group anchor must produce its own run_unit.

A single complaint group anchor may produce multiple run_unit records.

6.4 No blended run units

A run_unit must represent exactly one asserted website condition.

Blended run units and multi-assertion bundling are invalid.

6.5 Run-to-unit mapping rule

Each intake run must map to exactly one run_unit.

6.6 Unit-to-context execution rule

Each run_unit execution must occur under exactly one browser context per run.

6.7 Run unit normalization precedes execution

All run_unit records for a given matter must be created before any intake runs execute for that matter. A run_unit must not be created after the stop condition has been evaluated for that matter.

6.8 Unexecuted run units are not an error condition

The run cap may result in run_unit records that are fully normalized but never executed before the stop condition fires. Unexecuted run_unit records are not an error condition. They must remain in the manifest as normalized records. They must not be deleted, relabeled, or treated as invalid because no run was executed against them.

SECTION 7: OUTCOME LABEL SET AND DEFINITIONS [LOCKED]

7.1 Locked intake outcome label enum

Only these outcome labels are permitted:

- Observed as asserted
- Not observed as asserted
- Constrained
- Insufficiently specified for bounded execution

No additional labels, sublabels, variants, or aggregate labels are permitted.

7.2 Observed as asserted

Meaning: under bounded execution parameters for that run_unit and browser context, the asserted website condition manifested.

7.3 Not observed as asserted

Meaning: under bounded execution parameters for that run_unit and browser context, the asserted website condition did not manifest.

7.4 Constrained

Meaning: execution was blocked by an identifiable technical constraint that prevented a bounded attempt of the asserted condition, and that constraint maps to one locked constraint_class value.

7.5 Insufficiently specified for bounded execution

Meaning: the submitted materials do not provide sufficient bounded specification to attempt the asserted condition under controlled parameters for that run_unit and browser context.

7.6 One run, one outcome

Each run must produce exactly one outcome label from this locked set.

SECTION 8: EXECUTION CONTEXT RIGOR [LOCKED]

8.1 Contexts are parameter locks

Desktop and Mobile intake contexts are parameter locks, not descriptive approximations.

8.2 Replicated Desktop Browser Context baseline

Locked Desktop baseline ContextProfileRecord parameters:

- viewport_width: 1366
- viewport_height: 900
- zoom: 100%
- device_scale_factor: 1
- orientation: landscape
- is_mobile: false
- has_touch: false
  At Layer 1, these same parameters may be referenced using camelCase notation (deviceScaleFactor, isMobile, hasTouch) for human readability. The canonical field names for machine contracts remain the snake_case forms defined here and in Appendix A.

8.3 Replicated Mobile Browser Context baseline

Locked Mobile baseline ContextProfileRecord parameters:

- viewport_width: 393
- viewport_height: 852
- zoom: 100%
- device_scale_factor: 1
- orientation: portrait
- is_mobile: true
- has_touch: true
  At Layer 1, these same parameters may be referenced using

camelCase notation (deviceScaleFactor, isMobile, hasTouch) for human readability. The canonical field names for machine contracts remain the snake_case forms defined here and in Appendix A.

8.4 Peer-baseline scope rule

Desktop baseline and Mobile baseline are each a first-class intake baseline. Neither is subordinate to the other.

For generic website accessibility allegations, both Replicated Desktop Browser Context and Replicated Mobile Browser Context must be brought into scope under the intake determination logic.

Where the submitted materials expressly cabin the asserted condition to one baseline only, intake must issue the corresponding single-baseline determination.

Where one baseline is feasible and the peer baseline is blocked under controlled parameters, intake must issue the corresponding constrained-baseline determination.

Generic accessibility phrasing must not be treated as Desktop-only by default.

8.5 Reflow separation

Reflow testing under WCAG 1.4.10 is operationally separate from baseline intake capture.

8.6 Reflow primary method

- viewport_width: 320
- viewport_height: not fixed; determined by browser
- zoom: 100%
- orientation: portrait
- device_scale_factor: 1
- is_mobile: false
- has_touch: false

8.7 Reflow supplemental method

- viewport_width: 1280
- viewport_height: not fixed; determined by browser
- zoom: 400%

- effective_width: 320
- orientation: portrait
- device_scale_factor: 1
- is_mobile: false
- has_touch: false

8.8 Reflow does not alter baselines

Neither reflow method may alter the locked Desktop or Mobile intake baselines.

SECTION 9: MECHANICAL NOTE DISCIPLINE AND CONSTRAINT CLASSIFICATION [LOCKED]

9.1 Note permission gate

Notes are permitted only when:

- outcome_label = Constrained
- outcome_label = Insufficiently specified for bounded execution
- the matter determination is DETERMINATION: ELIGIBLE FOR DESKTOP TECHNICAL RECORD BUILD / MOBILE BASELINE: CONSTRAINED because Mobile baseline execution cannot be performed under controlled parameters when Mobile context is in scope
- the matter determination is DETERMINATION: ELIGIBLE FOR MOBILE TECHNICAL RECORD BUILD / DESKTOP BASELINE: CONSTRAINED because Desktop baseline execution cannot be performed under controlled parameters when Desktop context is in scope

Gates 1 and 2 authorize per-run notes, captured in MechanicalNoteRuleRecord (A6) with the corresponding run_id. Gates 3 and 4 authorize matter-level notes that are not tied to a single run; these are captured in the matter_level_note field of IntakeDeterminationRecord (A4). Gates 3 and 4 do not replace or duplicate per-run note authorization under gates 1 and 2.

9.2 Notes prohibited for

Notes are prohibited for:

- Observed as asserted
- Not observed as asserted
- statements about sufficiency reached or not reached

- statements about run ordering, context ordering, or interleaving decisions
- attempts to explain, defend, or argue outcome selection

9.3 Mechanical sentence rule

Any permitted note must be exactly one mechanical sentence stating the blocking condition only.

9.4 Note content restrictions

Permitted notes must contain:

- no legal terms
- no evaluative language
- no blame language
- no speculation
- no motive, severity, or probability framing

9.5 Locked `constraint_class` enum

If `outcome_label = Constrained`, the internal record must use exactly one of:

- `AUTHWALL`
- `BOTMITIGATION`
- `GEOBLOCK`
- `HARDCRASH`
- `NAVIMPEDIMENT`

9.6 No invented constraint classes

No additional `constraint_class` values are permitted.

9.7 Unmapped blocker rule

If a blocking condition does not map to one of the locked `constraint_class` values, the run must not be labeled `Constrained`. It must instead be labeled `Insufficiently specified for bounded execution`, with the mechanical note stating the missing bounded path or trigger.

9.8 Locked `note_basis` enum

If note_permitted = true, the note_basis field in the MechanicalNoteRuleRecord must use exactly one of the following values:

- outcome_constrained
- outcome_insufficiently_specified
- determination_desktop_eligible_mobile_constrained
- determination_mobile_eligible_desktop_constrained

No additional note_basis values are permitted.

SECTION 10: RUN SEQUENCING, STATE ISOLATION, AND TEMPORAL LOGGING [LOCKED]

10.1 Sequencing rule

When both Desktop and Mobile contexts are in scope, run order must alternate:

- Desktop
- Mobile
- Desktop
- Mobile

continuing until a stop condition is reached.

When only one baseline is in scope, all runs must execute in that baseline context and no interleaving occurs.

When both baselines are in scope but one baseline is blocked by a constraint, runs in the constrained context must still be physically attempted in their alternating slot. Each alternating slot for a constrained baseline requires a fresh physical attempt regardless of prior constraint outcomes for that context. The result of each such attempt must be recorded as a distinct IntakeRunRecord using the locked outcome label set and the applicable constraint_class. It consumes a run slot. The alternating sequence does not skip or collapse because one baseline is constrained.

10.2 Anti-manipulation sequencing rule

Run sequence must not be reordered, clustered, or manipulated to favor any asserted condition, context, or party.

10.3 Clean-state isolation between every run

Each run must execute in a fresh isolated browser context.

Storage state must not persist between runs. Storage isolation applies to all browser-managed state mechanisms, including:

- cookies
- localStorage
- sessionStorage
- IndexedDB
- Cache API
- service worker registrations

10.4 Clean-state failure handling

If clean-state isolation cannot be established for a run, a StateIsolationRecord must be generated for that run with fresh_browser_context = false and storage_state_persisted = true. The run must not proceed to condition evaluation.

If the isolation failure maps to an identifiable technical blocker that fits exactly one locked constraint_class value, the run must be labeled Constrained with the applicable constraint_class.

If the isolation failure does not map to any locked constraint_class value, the run must be labeled Insufficiently specified for bounded execution.

10.5 Temporal logging

Each run must record internal timestamps sufficient to show sequencing and separation, at minimum:

- run_start
- run_end

10.6 Temporal logging is internal only

These timestamps are internal execution metadata only. They must not be surfaced in any external-facing intake output and must not be indirectly signaled through language such as lengthy, brief, or extensive.

APPENDIX A: LOCKED MACHINE RECORD SCHEMAS FOR INTAKE

A1. IntakeRunRecord Schema

Required fields:

- matter_id
- run_id
- complaint_group_anchor_id
- run_unit_id
- context_id
- outcome_label
- constraint_class
- mechanical_note
- run_start_local
- run_start_epoch_ms
- run_end_local
- run_end_epoch_ms

Constraints:

- run_id must be unique per matter_id; no two IntakeRunRecord entries for the same matter_id may share the same run_id
- one run maps to exactly one run_unit_id
- one run maps to exactly one context_id
- context_id must resolve to a valid ContextProfileRecord defined in A7
- outcome_label must be one of the four locked intake outcome labels
- constraint_class must be empty string unless outcome_label = Constrained
- mechanical_note must be empty string unless note permission gate permits a note
- complaint_group_anchor_id must resolve to a valid ComplaintGroupAnchorRecord where anchor_type is one of the two locked forms defined in A2
- run timing fields are internal only

A2. ComplaintGroupAnchorRecord Schema

Required fields:

- matter_id

- complaint_group_anchor_id
- anchor_type
- anchor_value

Constraints:

- anchor_type must be one of:
  - page_paragraph_range
  - page_bullet_range
- anchor_value must be a string expressing the page-scoped range corresponding to the declared anchor_type; for page_paragraph_range it must identify the paragraph range by page and paragraph position; for page_bullet_range it must identify the bullet range by page and bullet position

A3. RunUnitRecord Schema

Required fields:

- matter_id
- run_unit_id
- complaint_group_anchor_id
- asserted_condition_text
- desktop_in_scope
- mobile_in_scope
- created_context_basis

Constraints:

- one run_unit_id represents exactly one asserted website condition
- one complaint group anchor may map to multiple run units
- no blended or multi-assertion run units allowed
- desktop_in_scope must be boolean
- mobile_in_scope must be boolean
- at least one of desktop_in_scope or mobile_in_scope must be true; any run_unit where both are false must be rejected and must not have any intake runs executed against it
- desktop_in_scope and mobile_in_scope reflect the peer-baseline scope rule defined in Section 8.4; both must be true for generic accessibility allegations unless the submitted materials expressly cabin scope to one baseline
- created_context_basis must be exactly one of the following locked values:
  - generic_accessibility_allegation — both baselines brought into scope because the submitted materials assert a generic accessibility condition not expressly cabined to one baseline

- materials_cabined_desktop_only — Desktop baseline only, because the submitted materials expressly limit the asserted condition to Desktop
- materials_cabined_mobile_only — Mobile baseline only, because the submitted materials expressly limit the asserted condition to Mobile
- constrained_peer_baseline — one baseline in scope and feasible, peer baseline in scope but blocked under controlled parameters

A4. IntakeDeterminationRecord Schema

Required fields:

- matter_id
- determination_template
- generated_at_local
- generated_at_epoch_ms
- matter_level_note

Constraints:

- determination_template must be one of the eight locked determination templates
- exactly one IntakeDeterminationRecord may exist per matter_id
- no paraphrase
- no modifiers
- no appended explanatory leakage; for purposes of this constraint, appended explanatory leakage means any text added after the locked determination template string that provides rationale, context, qualification, process detail, or any other content not part of the locked template
- matter_level_note must be empty string unless the matter determination is Template 3 or Template 5 and a matter-level note is authorized under Section 9.1 gates 3 or 4
- when present, matter_level_note must be exactly one mechanical sentence stating the blocking condition only and must comply with all note content restrictions defined in Section 9.4

A5. IntakeManifestRecord Schema

Required fields:

- matter_id

- spec_version
- scope_anchor_reference
- intake_runs
- run_units
- complaint_group_anchors
- determination_record
- internal_timing_metadata

Constraints:

- every run reference must resolve
- every run unit reference must resolve
- every complaint group anchor reference must resolve
- determination record must resolve
- internal-only metadata must remain non-external
- spec_version must identify the Layer 3 document version under which this manifest was generated
- scope_anchor_reference must identify the complaint or demand materials that served as the scope anchor; it must contain sufficient reference information to trace back to the submitted materials and is not externally surfaced
- matter_id format is governed externally; syntax validation of matter_id is not performed at this layer
- intake_runs must be an ordered list of references to IntakeRunRecord, where order corresponds to actual execution sequence
- run_units must be a list of references to RunUnitRecord covering all normalized run units for the matter, including unexecuted units
- complaint_group_anchors must be a list of references to ComplaintGroupAnchorRecord covering all anchor records for the matter

- determination_record must be a single reference to exactly one IntakeDeterminationRecord for the matter

- internal_timing_metadata must be a structured internal-only object sufficient to preserve timing resolution for all runs in the manifest; it must not be externally surfaced in any form

A6. MechanicalNoteRuleRecord Schema

Required fields:

- matter_id
- run_id
- note_permitted — boolean
- note_basis

Constraints:

- note_permitted may be true only under the locked note-permission gate defined in Section 9.1
- when note_permitted = false, note_basis must be empty string
- when note_permitted = true, note_basis must be exactly one of the four locked note_basis values defined in Section 9.8
- no notes for Observed as asserted or Not observed as asserted runs

A7. ContextProfileRecord Schema

Required fields:

- context_id
- viewport_width
- viewport_height
- orientation
- zoom
- device_scale_factor
- is_mobile
- has_touch

Constraints:

- context_id must be one of:
  - desktop_baseline
  - mobile_baseline
  - reflow_primary
  - reflow_supplemental

- viewport_width must match the locked pixel value for the declared context_id as defined in Sections 8.2, 8.3, 8.6, and 8.7
- viewport_height must match the locked pixel value for the declared context_id where a fixed value is defined; for reflow_primary and reflow_supplemental, viewport_height is not a fixed locked value and must not be validated against a pixel constraint
- zoom is the canonical locked field name for all four context_id values
- for desktop_baseline and mobile_baseline, all required fields must exactly match the locked parameter values defined in Sections 8.2 and 8.3 respectively
- for reflow_primary and reflow_supplemental, viewport_height is conditionally non-fixed; all other fields must match the values defined in Sections 8.6 and 8.7 respectively
- baseline values must match the locked intake doctrine exactly

A8. SequencingRecord Schema

Required fields:

- matter_id
- run_sequence
- both_baselines_in_scope
- stop_basis

Constraints:

- when both baselines are in scope, run sequence must alternate Desktop and Mobile until stop
- when only one baseline is in scope, all runs must execute in that baseline context
- run_sequence must be an ordered array of context_id values representing the actual execution order for the matter, where each element corresponds to one run in the sequence
- stop_basis must be either:
  - sufficiency_reached
  - run_cap_reached
- when both stop conditions occur on the same run — sufficiency is reached on the run that also completes the 10-run cap — stop_basis must be recorded as sufficiency_reached
- both_baselines_in_scope must be boolean

A9. StateIsolationRecord Schema

Required fields:

- matter_id
- run_id
- fresh_browser_context
- storage_state_persisted

Constraints:

- for runs where clean-state isolation was successfully established: fresh_browser_context must be true and storage_state_persisted must be false
- for runs where clean-state isolation could not be established, as governed by Section 10.4: fresh_browser_context must be false and storage_state_persisted must be true; the run must not proceed to condition evaluation and no outcome label other than the applicable locked label may be assigned
- storage isolation must cover all browser-managed state mechanisms defined in Section 10.3, including cookies, localStorage, sessionStorage, IndexedDB, Cache API, and service worker registrations

A10. ExternalOutputValidationRecord Schema

Required fields:

- matter_id
- matter_level_note_compliance_check_passed
- spec_version
- determination_template_used
- forbidden_disclosure_check_passed
- forbidden_language_check_passed
- mandatory_term_check_passed
- matter_level_context_disclosure_check_passed
- per_run_context_leakage_check_passed
- indirect_signaling_check_passed
- functional_equivalent_review_flagged
- functional_equivalent_review_cleared
- anti_hedging_review_flagged
- anti_hedging_review_cleared

Constraints:

- all automated pass/fail checks must pass for external-facing intake output to be valid; any record where functional_equivalent_review_flagged = true or anti_hedging_review_flagged = true requires reviewer clearance

before external release; the corresponding _cleared field must be true before external output is permitted

- spec_version must identify the Layer 3 document version governing this validation run
- determination_template_used must be one of the eight locked intake determination templates
- forbidden_disclosure_check_passed must verify that no output discloses any item prohibited under Section 1.4 or Section 4.3
- forbidden_language_check_passed must verify that no output contains any term or framing prohibited under Sections 3.1, 3.3, or 3.4
- mandatory_term_check_passed must verify that when external-facing output describes scope and context, all five exact terms from Section 3.2 are present and no paraphrase is used
- matter_level_context_disclosure_check_passed must verify that any context disclosure uses only the locked exact terms Replicated Desktop Browser Context and Replicated Mobile Browser Context at the matter level only
- per_run_context_leakage_check_passed must verify that no per-run sequencing, per-run context assignment, or per-asserted-condition context detail is present in any external-facing output
- indirect_signaling_check_passed covers only the exact prohibited phrases listed in Section 3.5; it does not cover functional equivalents of those phrases; functional-equivalent coverage is governed exclusively by functional_equivalent_review_flagged; indirect_signaling_check_passed must verify that none of the exact listed phrases appear in the output
- functional_equivalent_review_flagged must be set to true when the output contains any phrasing that requires reviewer judgment to assess for indirect-signaling equivalence under Section 3.5; this check is not fully automatable and must be routed for reviewer clearance before external release
- functional_equivalent_review_cleared must be true only after a reviewer has confirmed no indirect-signaling functional equivalent is present; it must default to false and may only be set to true by reviewer action
- anti_hedging_review_flagged must be set to true when the output contains any phrasing that requires reviewer judgment to assess for hedging functional equivalents under Section 4.4; this check is not fully automatable and must be routed for reviewer clearance before external release
- anti_hedging_review_cleared must be true only after a reviewer has confirmed no hedging functional equivalent is present; it must default to false and may only be set to true by reviewer action

- `matter_level_note_compliance_check_passed` must verify that when `matter_level_note` is present in the `IntakeDeterminationRecord`, it contains exactly one mechanical sentence, states only the blocking condition, and contains no content prohibited under Section 9.4; when `matter_level_note` is empty string, this check must pass automatically
- any prohibited language, prohibited disclosure, or per-run leakage must fail validation and block external output